

purple_air_DL File Guide

A file-by-file account of the repository: low-cost PurpleAir PM2.5 sensors in the City of Los Angeles are calibrated against an EPA reference monitor. The calibrated readings are then spread across census tracts to estimate excess deaths from the January 2025 wildfire smoke. This page is based on reading every script and inspecting every data file. Nothing in the folder was changed.

4 top-level folders + `.git` **37** Python scripts (plus 21 inside zips)

260 per-sensor CSVs in LA-city-files **137** sensor download folders **26** figures

Study window **2025-01-01 → 02-28**, fire **Jan 7-31**

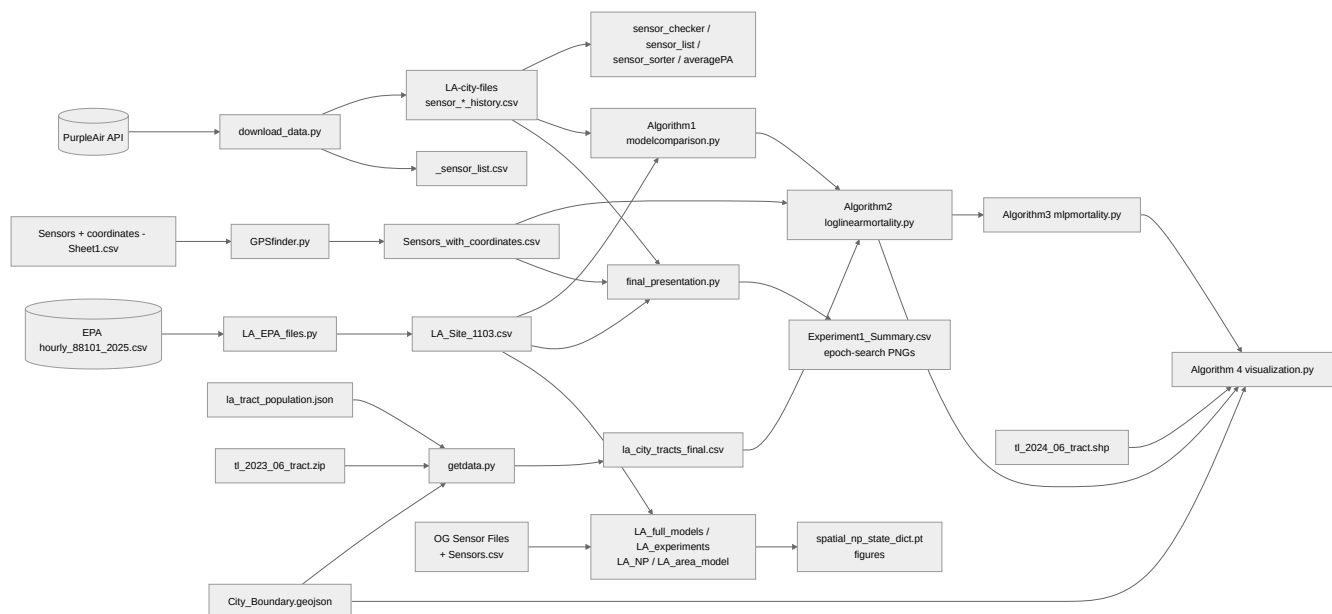
Overall summary

The folder is a research project, apparently for a thesis, that grew in stages. Its layers are:

- **Raw data.** Hourly PurpleAir readings for LA sensors ([LA-city-files](#): 260 city sensors; [OG Sensor Files](#): an earlier 137-sensor set). There are also EPA reference-monitor files: LA site 1103, plus Illinois site 4002 for early tests. Census tract population and geometry and the LA city boundary complete it.
- **Data-prep utilities.** Scripts that download sensors, look up coordinates, build the tract table, and check, sort and average the sensor files.
- **Calibration experiments.** A series of scripts that predict the EPA reading from PurpleAir PM2.5 plus temperature, humidity and pressure. They start with plain linear regression on one Illinois week. They move on to log-linear models, MLPs and a spatial neural process on the LA network, and end with the “final presentation” comparison of the closest sensor vs sensors within 3 km and 5 km.
- **The mortality pipeline.** It exists in two parallel versions. The standalone `Algorithm1-4` scripts in `code1` calibrate with sensor 208493, map PM2.5 to every tract with a thin-plate spline and estimate excess deaths (log-linear and MLP), then draw maps. The modular package in `final_experiment.zip` does the same in five stages.

How the files connect

Main data flow. Boxes are files; arrows point from inputs to the scripts that read them and on to their outputs.



Early calibration tests

Site_4002.csv (Illinois EPA) +
 sensor_data.csv / 164317...csv →
 chat_linreg,
 linear/loglinear_regression_model,
 first_machine_model, presentation_code,
 week2_draftcode → diagnostic plots,
 mlp_vs_log_lin.png .

LA 137-sensor experiments

Sensors.csv + per-sensor folders +
 LA_Site_1103.csv → LA_full_models,
 LA_experiments, LA_NP, LA_area_model →
 sensors_by_distance.csv , MSE/epoch
 plots, NP weights.

Final presentation

LA-city-files +
 Sensors_with_coordinates.csv + EPA →
 final_presentation(2) →
 Experiment1_Summary.csv , epoch-search
 PNGs.

Standalone mortality pipeline

Algorithm 1 → outputs/calibration ,
 outputs/checkpoints → Algorithm 2 →
 Algorithm 3 → Algorithm 4 maps.
 EPA_mortality_estimate.py is a simple
 EPA-only benchmark.

Modular mortality pipeline

final_experiment.zip : stage1 → stage5
 via checkpoints/. Algorithm1.py ,
 config.py , Data utils.py and Run real
 data pipeline.py in code1 are loose
 copies from it.

Things to know

These came up while reading the files. None of them was changed.

A PurpleAir API key is written into the code.

`download_data.py` and `GPSfinder.py` contain the same key, and the repository has a GitHub remote (`geethurajan/purple_air_DL`). If that repository is public, the key is visible to anyone.

Scripts expect all inputs in one folder, but the inputs are spread across folders.

For example, Algorithm 2 needs `Sensors_with_coordinates.csv` (code2) and the sensor files (LA-city-files) next to it. Algorithm 4 needs `tl_2024_06_tract.shp` (code2) and `City_Boundary.geojson` (code1).

Some imports point to files that are not loose in the folder.

`Algorithm1.py` and `Run real data pipeline.py` import modules that exist only inside `final_experiment.zip`. `Data utils.py` has a space in its name, so `import data_utils` won't find it. In the zip, `run_pipeline.py` imports `algorithm4_visualization`, which the zip does not contain.

Two Algorithm 1 scripts write incompatible outputs.

Only `Algorithm1 modelcomparison.py` writes the coefficient names and checkpoint paths that Algorithms 2 and 3 read. `algorithm1_modelcomparison.py` uses different names and locations.

Folder and column names don't always match.

The `LA_*` scripts look for `PurpleAir Download Sensor N` folders; the downloads here are in `OG Sensor (N)`, where N is the sensor number plus one. `Phase1.py` expects `time / pm2.5_atm` columns, and `final_presentation2.py` expects `timestamp`; the LA-city-files use `time_stamp / pm2.5_cf_1`.

Referenced files that are not in the folder:

`hourly_88101_2025.csv`, `Sensors - Sheet1.csv`, `purple_air_metadata.csv`,
`raw_la_tract_population.csv`, `sensor_metadata.csv`, `mock_sensors/`, `purpleair_sensors/`.

Comments that disagree with the code.

`Algorithm1 modelcomparison.py`'s docstring describes a 128-64-32 network, but it builds a 16-unit one. Algorithm 2's header says ΔPM is the PM value itself, but the code subtracts the February baseline. `maxmin.py` prints "pm2.5_cf_1" for humidity. `LA_full_models.py` scores its MLP on training data while labelling it "n_test".

Fire and baseline windows differ between scripts.

Algorithms 2-3: fire Jan 7-31 vs Feb 1-28. `Phase1.py`: Jan 7-27 vs Jan 28 onward.

`EPA_mortality_estimate.py`: Jan 7-31 vs the rest of Jan 1-Mar 1. The modular pipeline's config still has July 2025 vs Aug 1-20, 2025.

Git shows 310 files as modified.

The only difference is line endings (Windows CRLF vs LF); the contents are the same as the last commit.

code1_zip_short 73 files

The main working folder: scripts from every stage of the project, the EPA file for LA, the tract table and boundary, result CSVs, figures and three zip archives.

Getting and preparing the data 11

download_data.py

PYTHON SCRIPT

Contains

PurpleAir API client with retry/back-off, a city-polygon filter (shapely), and a hard-coded PurpleAir API key.

Purpose

Lists outdoor PurpleAir sensors inside a bounding box around LA (33.70–34.34°N, –118.68 to –118.15°W), keeps only those inside the LA city polygon (so enclaves like Beverly Hills are excluded), then downloads hourly averages of `pm2.5_cf_1`, humidity, temperature and pressure for 2025-01-01 to 2025-03-01 (Pacific time) for each sensor.

Inputs

PurpleAir API; `City_Boundary.geojson`.

Outputs

`_sensor_list.csv` and one `sensor_<id>_history.csv` per sensor that returned data, written next to the script. Sensors with no data get no file.

Relation

Source of the files in **LA-city-files** (the 192 non-empty files use exactly this script's column order) and of `_sensor_list.csv`.

GPSfinder.py

PYTHON SCRIPT

Contains

Short PurpleAir API script with a hard-coded API key.

Purpose

Looks up latitude/longitude for a list of sensor IDs, 100 at a time, and fills them into the table.

Inputs

Sensors + coordinates - Sheet1.csv (in code2).

Outputs

Sensors_with_coordinates.csv (in code2).

Relation

Its output is the coordinate table used by final_presentation.py, final_presentation2.py and Algorithm2 loglinearmortality.py.

getLApoints.py

PYTHON SCRIPT

Contains

About 30 lines; reads the city boundary.

Purpose

Prints the northernmost, southernmost, easternmost and westernmost coordinates of the LA city boundary.

Inputs

City_Boundary.geojson.

Outputs

Console output only.

Relation

Gives the kind of extent numbers used for the bounding box in download_data.py.

getdata.py

PYTHON SCRIPT

Contains

Hand-written shapefile (.shp/.dbf) readers, a shoelace-formula centroid calculation and a ray-casting point-in-polygon test (no GIS libraries).

Purpose

Builds the census-tract table: loads tract population, reads 2023 California tract polygons straight from the zip, keeps LA County (COUNTYFP 037), computes each tract's centroid, keeps tracts whose centroid is inside the city boundary, and adds local x/y km coordinates relative to the EPA monitor (34.06659, -118.22688).

Inputs

la_tract_population.json , tl_2023_06_tract.zip (in code2), City_Boundary.geojson .

Outputs

la_city_tracts_final.csv .

Relation

Its output feeds every mortality script (EPA-only estimate, Algorithms 2-4, the modular pipeline).

LA_EPA_files.py

PYTHON SCRIPT

Contains

Chunked CSV reader (1 million rows at a time).

Purpose

Pulls the rows for one monitoring site out of EPA's national hourly PM2.5 file, filtering on State Code 6 (California) and a Site Num set at the top of the script (currently 30). It does not filter on county.

Inputs

hourly_88101_2025.csv (EPA national file; **not in the folder**).

Outputs

LA_Site_<n>.csv .

Relation

The output naming matches LA_Site_1103.csv , the EPA file used throughout.

file_mover.py

PYTHON SCRIPT

Contains

About 30 lines.

Purpose

For every folder named `PurpleAir Download Sensor ...` next to the script, moves its single CSV up one level and deletes the folder.

Inputs

Folders named `PurpleAir Download Sensor N` (none exist now; the downloads in this repo are in `OG Sensor (N)` folders).

Outputs

Moved CSV files; deleted folders.

Relation

Housekeeping for the per-sensor downloads that **OG Sensor Files** holds.

averagePA.py

PYTHON SCRIPT

Contains

Loader plus one matplotlib time-series plot.

Purpose

Averages `pm2.5_cf_1` across all sensor files for each timestamp (first averaging duplicates within a sensor), and counts how many sensors reported.

Inputs

All `sensor*_history.csv` files in the script's folder.

Outputs

`average_pm25_by_timestamp.csv` and `average_pm25_by_timestamp.png` (both present).

Relation

Run on the LA-city-files sensor set.

findwildfire.py

PYTHON SCRIPT

Contains

Two matplotlib plots. The docstring calls it `plot_epa_pm25.py`.

Purpose

Plots EPA hourly and daily-average PM2.5 for 2025-01-01 to 2025-03-01 to spot the wildfire period.

Inputs

`LA_Site_1103.csv`.

Outputs

`epa_pm25_hourly.png`, `epa_pm25_daily_avg.png`, `epa_pm25_daily_avg.csv` (all present).

Relation

The spike it shows (around Jan 8-10) matches the Jan 7 fire start date used by the mortality scripts.

maxmin.py

PYTHON SCRIPT

Contains

Nine lines.

Purpose

Prints the minimum and maximum of the `humidity` column of one sensor file. The print labels say `pm2.5_cf_1`, which does not match the column actually used.

Inputs

`sensor_94397_history.csv` (in LA-city-files).

Outputs

Console output only.

Relation

Quick check; nothing depends on it.

LA_models.py

PYTHON SCRIPT

Contains

About 40 lines; an early loader.

Purpose

Reads a sensor list, opens each sensor's CSV from its `PurpleAir Download Sensor N` folder, adds sensor number/ID/lat/lon, concatenates them and prints the head and column names.

Inputs

`Sensors - Sheet1.csv` (**not in the folder**) and `PurpleAir Download Sensor N/<id> 2025-01-01 2025-03-01 60-Minute Average.csv`.

Outputs

Console output only.

Relation

Uses the same per-sensor file names as **OG Sensor Files**, but a different folder name. It's an early version of the loader in the `LA_*` scripts.

Contains

Logging-based staging script with functions for EPA cleaning, PurpleAir cleaning, scenario averaging and population staging.

Purpose

Cleans the EPA series (120-hour completeness check, linear interpolation), loads PurpleAir files with a metadata table, drops sensors under 120 hours, then computes per-station mean PM2.5 for a “factual” window (Jan 7–27) and a “counterfactual” window (Jan 28 onward). It also stages tract population with a daily baseline mortality rate of 780/365/100,000.

Inputs

`LA_Site_1103.csv`, `sensor*_history.csv`, `purple_air_metadata.csv` and `raw_la_tract_population.csv` (the last two are **not in the folder**). It expects columns `time` and `pm2.5_atm`, which the LA-city-files sensor files do not have (they use `time_stamp` and `pm2.5_cf_1`).

Outputs

`data_staged/sensor_scenarios_averaged.csv`,
`data_staged/clean_population_baseline.csv` (not present).

Relation

An earlier take on the Algorithm 1 data stage; its fire/baseline windows differ from those in Algorithms 2–3.

Early calibration tests (Illinois site, July 2025) ⁵

chat_linreg.py

PYTHON SCRIPT

Contains

Pure-csv loader plus scikit-learn linear regression.

Purpose

Matches EPA and PurpleAir hours for July 1–7, 2025 (using EPA GMT times against the sensor's UTC “Z” times) and fits EPA PM2.5 = $f(\text{pm2.5_cf_1}, \text{humidity}, \text{pressure}, \text{temperature})$. Prints coefficients, R^2 , adjusted R^2 and every matched row.

Inputs

Site_4002.csv (in code2), 164317 2025-07-01 2025-07-08 60-Minute Average.csv .

Outputs

Console output only.

Relation

First calibration test; later reworked in linear_regression_model.py and first_machine_model.py .

linear_regression_model.py

PYTHON SCRIPT

Contains

Linear regression plus regression diagnostics (scipy, statsmodels).

Purpose

Fits EPA PM2.5 on raw PurpleAir PM2.5, humidity, pressure and temperature for July 1–7, 2025, then shows a Q-Q plot, residuals vs predicted, a scale-location plot, residuals vs leverage (Cook's distance) and four single-variable scatter plots. EPA and sensor values are paired by row order, not by timestamp.

Inputs

Site_4002.csv (local time), sensor_data.csv (both in code2).

Outputs

Plots shown on screen; saves y_vs_individual_variables.png (present in code2).

Relation

The acms_surp_*.png figures without a “2” appear to be this script's on-screen plots, saved by hand (residuals are on the $\mu\text{g}/\text{m}^3$ scale).

loglinear_regression_model.py

PYTHON SCRIPT

Contains

Same structure as linear_regression_model.py.

Purpose

Log-linear version: $\ln(\text{EPA})$ on $\ln(\text{PurpleAir PM}_{2.5})$, humidity, pressure, temperature (positive values only), with the same diagnostic plots.

Inputs

Site_4002.csv , sensor_data.csv .

Outputs

Plots shown on screen; saves logy_vs_individual_variables.png (present).

Relation

The acms_surp_*2.png figures appear to be this script's plots (residuals on the log scale).

first_machine_model.py

PYTHON SCRIPT

Contains

pandas + scikit-learn.

Purpose

Merges EPA (GMT) and PurpleAir on timestamp, fits a standardized log-linear regression on an 80/20 split, and reports RMSE, R^2 , adjusted R^2 and coefficients.

Inputs

Site_4002.csv , sensor_data.csv .

Outputs

Console output only.

Relation

Its first half is repeated in week2_draftcode.py (code2), which adds two neural models.

presentation_code.py

PYTHON SCRIPT

Contains

pandas + scikit-learn + matplotlib.

Purpose

For July 1-7, 2025 it lists unmatched timestamps, then fits three models: standard linear, log-linear (in-sample) and an “ML log-linear” fitted on a 50/50 split. It plots EPA against all three model predictions over time.

Inputs

Site_4002.csv , sensor_data.csv .

Outputs

Plot shown on screen only.

Relation

comparisons.png has this plot's exact title; model_comparison.png is a near-identical version titled “Hourly Averages”, probably from an earlier edit.

LA sensor-network experiments 2

LA_full_models.py

PYTHON SCRIPT

Contains

Header notes (137 sensors; some indoor or bought after the fires; a train station near a cluster), a hard-coded list of excluded sensor numbers, OLS models, outlier detection and an MLP.

Purpose

Computes each sensor's distance to the EPA monitor, loads the sensors not excluded (the current list keeps 12 of 137), merges with EPA by hour and builds 1-hour lag features only where the previous row is exactly one hour earlier. It then fits four OLS models of $\ln(\text{EPA})$: full and reduced, each using `pm2.5_atm` or `pm2.5_cf_1`. It flags outlier sensors (PM2.5 unusual versus neighbours within 0.8 km, or unusually low R^2) and trains an MLP (64-32-16). The MLP is scored on its training data; an inline comment says this was changed from the test set, but the printout still says “n_test”.

Inputs

`Sensors.csv`, `LA_Site_1103.csv`, `PurpleAir Download Sensor N/*.csv` folders.

Outputs

`sensors_by_distance.csv` plus console tables.

Relation

The `sensors_by_distance.csv` in code2 (125 EXCLUDED) matches this script's current exclusion list.

LA_experiments.py

PYTHON SCRIPT

Contains

Same loading and feature steps as LA_full_models.py, with a shorter exclusion list.

Purpose

Experiment 3 (run first): average MLP test MSE over 50 random splits for 50-750 epochs, and picks the best. Experiment 1: in-sample R^2 for the four OLS models and the MLP. Experiment 2: average test R^2 over 50 splits of 80/20. It then prints them side by side.

Inputs

Sensors.csv , LA_Site_1103.csv , PurpleAir Download Sensor N folders.

Outputs

experiment3_mse_vs_epochs.png (present) and console tables.

Relation

The three Average Test MSE vs Epoch (version N).png files look like earlier runs of Experiment 3 (version 1 includes a 1,000-epoch point that the current list does not have).

Spatial neural-process models ²

LA_NP.py

PYTHON SCRIPT

Contains

PyTorch spatial conditional neural process: a set encoder and a decoder that outputs a mean and log-variance, trained with Gaussian negative log-likelihood and early stopping.

Purpose

Uses only sensor locations (km relative to the EPA site) and log PM2.5 (`pm2.5_cf_1`). Each hour becomes a training episode: some sensors are context, the rest are targets. EPA data is used only at the end, to score the model's prediction at the EPA location with the metrics of Shen, Crippa & Castruccio (2021): R^2 , MAE, NMBF, NMAEF and IOA. It uses the same long exclusion list as `LA_experiments.py`.

Inputs

`Sensors.csv` , `LA_Site_1103.csv` , `PurpleAir Download Sensor N` folders.

Outputs

`spatial_np_state_dict.pt` (the copy in code2 matches this architecture) and `np_predicted_vs_epa.png` .

Relation

The base version of `LA_area_model.py` .

LA_area_model.py

PYTHON SCRIPT

Contains

Same model as LA_NP.py.

Purpose

Same pipeline with no sensors excluded, up to 400 epochs and patience 40. Adds a diagnostic of which EPA hours are missing, and a calibration step that regresses $\ln(\text{EPA})$ on $\ln(\text{NP prediction})$ plus temperature, pressure and humidity from the sensor nearest the EPA site. It also compares that fit with a fixed set of hard-coded coefficients.

Inputs

Sensors.csv , LA_Site_1103.csv , PurpleAir Download Sensor N folders.

Outputs

spatial_np_state_dict.pt , np_predicted_vs_epa.png (both overwrite LA_NP.py's),
epa_missing_diagnostic.png .

Relation

calibration_before_after.png shows this before/after-calibration idea, but no script in the folder saves a file with that name.

Final-presentation experiments ²

final_presentation.py

PYTHON SCRIPT

Contains

PyTorch MLP (64-32), statsmodels OLS, 5-fold epoch search.

Purpose

Loads every sensor file with 1-hour lags, attaches coordinates, converts EPA local time to UTC and merges on timestamp. It computes distance to the EPA monitor, uses log1p for both PM2.5 values and drops sensors whose mean is more than 3 SD from the average. It builds three datasets: the closest sensor, all sensors within 3 km, and all within 5 km. For each it searches epochs 5–200, then runs Experiment 1 (fit and evaluate on all data) and Experiment 2 (50 random 80/20 splits). The file ends inside the Experiment 2 loop: the results are collected but never summarised or saved.

Inputs

LA_Site_1103.csv , Sensors_with_coordinates.csv (code2), sensor*_history.csv .

Outputs

closest_epoch_search.png , 3km_epoch_search.png , 5km_epoch_search.png ,
Experiment1_Summary.csv (all present).

Relation

Superseded by final_presentation2.py .

final_presentation2.py

PYTHON SCRIPT

Contains

Variant of final_presentation.py.

Purpose

Differences: it expects a column named `timestamp` in each sensor file (the actual files use `time_stamp`, so the column check would skip them). It adds an “all sensors” dataset, drops the outlier-sensor removal and the EPA date filter, and redoes the epoch search inside every Experiment 2 replicate using training data only. It finishes Experiment 2 with a summary table.

Inputs

Same as final_presentation.py.

Outputs

Epoch-search PNGs, `Experiment1_Summary.csv`,
`Experiment2_Summary_Full_Epoch_Search.csv` (not present).

Relation

Newer version of `final_presentation.py`.

Mortality pipeline, standalone version (Algorithms 1 to 4) ⁶

Algorithm1 modelcomparison.py

PYTHON SCRIPT

Contains

Logging, checkpointing, TensorFlow/Keras MLP, six figure functions.

Purpose

Step 1 of the mortality pipeline. Pairs EPA with PurpleAir sensor **208493** for 2025-01-01 to 02-28, adds 1-hour lags, fits the log-linear calibration on all the data, and compares OLS with an MLP over 50 random 80/20 splits. The header says row-wise random splits replaced the originally specified blocked time splits. The docstring describes a 128-64-32 network, but the code builds Dense(16) → Dropout(0.2) → Dense(1).

Inputs

LA_Site_1103.csv , sensor_208493_history.csv .

Outputs

outputs/calibration/loglinear_coefficients.csv , model_comparison.csv ,
cv_results.csv , algorithm1_summary.json ;
outputs/checkpoints/synchronized_calibration_data.csv ; figures and logs under
outputs/ .

Relation

Its coefficient names and checkpoint paths are exactly what **Algorithm 2** and **Algorithm 3** read.

Contains

An alternative Algorithm 1 (statsmodels + Keras 128-BatchNorm-64-32 MLP, seaborn).

Purpose

Loads all sensor files (keeping those with 120+ rows), picks sensor 208493, adds lags, fits the log-linear model, and runs 50 random-split comparisons. Here the MLP predicts raw EPA PM2.5 from raw PurpleAir PM2.5. PurpleAir timestamps keep their UTC offset while EPA timestamps are naive, which may stop the two from merging. `COORD_FILE` is defined but never used.

Inputs

`LA_Site_1103.csv`, all `sensor*_history.csv`.

Outputs

Under `outputs/calibration/`: `loglinear_coefficients.csv` (terms `const`, `log_pm25`, ...), `random_cv_results.csv`, `calibrated_training_predictions.csv`, `calibration_dataset.csv`, `synchronized_calibration_data.csv`, figures.

Relation

Not compatible with Algorithm 2. It writes different coefficient names (`const` vs `intercept`, `log_pm25` vs `log_pa_pm`) and puts the synchronized data in `calibration/` instead of `checkpoints/`.

Contains

Loaders, a thin-plate-spline surface (scipy `RBFIInterpolator`, smoothing 1e-3), mortality maths, a 50-replicate bootstrap, and the EPA-only comparison.

Purpose

Applies the Algorithm 1 coefficients to every sensor with at least 120 hours and averages each sensor to daily PM2.5. It fits a spline surface for each day and reads it at each tract centroid. For each tract and fire day (Jan 7–31), $\Delta\text{PM} = \text{PM} \text{ minus that tract's Feb 1–28 average}$. Then $\text{RR} = \exp(\gamma\Delta\text{PM})$, $\text{AF} = (\text{RR}-1)/\text{RR}$, and $\text{daily deaths} = \text{AF} \times 780/100,000/365 \times \text{population}$, summed over the fire days. Uncertainty combines the γ confidence interval (U) with 50 bootstrap refits (B): $T = U + B$. It also computes a spatially uniform EPA-only estimate. The header docstring says ΔPM is the PM value itself, but the code subtracts the February baseline.

Inputs

`outputs/calibration/loglinear_coefficients.csv` and
`outputs/checkpoints/synchronized_calibration_data.csv` (from Algorithm 1),
`LA_Site_1103.csv`, `Sensors_with_coordinates.csv`, `la_city_tracts_final.csv`,
`sensor*_history.csv`.

Outputs

Under `outputs/mortality/`: `daily_calibrated_pm_by_sensor.csv`,
`daily_tract_pm_loglinear.csv`, `bootstrap_mortality_replicates.csv`,
`tract_mortality_loglinear.csv`, `epa_only_mortality.csv`, `algorithm2_summary.json`. Also
`outputs/checkpoints/sensor_design_matrix.csv`.

Relation

Reads Algorithm 1's output. Its design matrix is read by Algorithm 3, and its mortality tables by Algorithm 4.

Contains

Keras MLP (Dense 16 → Dropout 0.2 → Dense 1), no fixed random seeds, the same spline and mortality code as Algorithm 2.

Purpose

Trains a freshly initialised MLP 50 times on the synchronized calibration data (standardized). Each time it applies the MLP to all sensors, builds the daily spline surface and computes tract mortality with its γ -interval variance. The runs are combined as mean M, total variance = $\text{mean}(U) + \text{Var}(M)$, and a 95% CI.

Inputs

outputs/checkpoints/synchronized_calibration_data.csv (Algorithm 1),
outputs/checkpoints/sensor_design_matrix.csv (Algorithm 2),
la_city_tracts_final.csv .

Outputs

outputs/mortality/mlp_mortality_M_by_iteration.csv ,
mlp_mortality_U_by_iteration.csv , tract_mortality_mlp.csv , algorithm3_summary.json .

Relation

Must run after Algorithms 1 and 2; feeds Algorithm 4.

Algorithm 4 visualization.py

PYTHON SCRIPT

Contains

geopandas choropleth maps and diagnostic charts at 300 DPI.

Purpose

Draws tract maps (clipped to the city outline) of excess deaths per 100,000 for the log-linear and MLP results, and maps of each minus the EPA-only estimate. Also draws histograms of tract mortality and fire-period PM2.5, a spline diagnostic for a mid-fire day, and a bar chart of the calibration coefficients.

Inputs

Algorithm 2 and 3 outputs, `tl_2024_06_tract.shp` (code2, with its .shx/.dbf/.prj), `City_Boundary.geojson`, `la_city_tracts_final.csv`.

Outputs

`outputs/figures/fig1_mortality_loglinear.png`, `fig2_mortality_mlp.png`, `fig3_diff_loglinear_vs_epa.png`, `fig4_diff_mlp_vs_epa.png`, histograms, diagnostic and coefficient charts (none present).

Relation

Last step of the standalone pipeline.

EPA_mortality_estimate.py

PYTHON SCRIPT

Contains

Short, self-contained calculation.

Purpose

City-wide excess mortality from the EPA monitor alone. It averages daily PM2.5 for fire days (Jan 7–31) and non-fire days (the rest of Jan 1–Mar 1), takes the difference, and computes RR, AF and $deaths = 780/100,000 \times \text{total population} \times AF$, with CI bounds from γ . Unlike Algorithms 2–3, it applies the annual rate once instead of converting to a daily rate and summing over fire days. It also prints a daily summary table.

Inputs

`LA_Site_1103.csv`, `la_city_tracts_final.csv`.

Outputs

Console output only.

Relation

A simple benchmark next to the EPA-only estimate in Algorithm 2. The `daily_pm25.csv` columns match the daily table it builds, but the script does not save it.

Mortality pipeline, modular version (copied from final_experiment.zip)

4

config.py

PYTHON MODULE

Contains

Constants only: replicate counts (50), feature names, MLP settings (128-64-32, dropout 0.3), $z = 1.96$, spline $\lambda = 1e-3$, the LA projection reference point, $\gamma = 0.00094$ (0.00073–0.00116, Atkinson et al. 2014), and a baseline mortality of 780 per 100,000 per year (GBD 2017).

Purpose

Shared settings for the modular pipeline.

Relation

The same as `config.py` inside `final_experiment.zip`. It's imported by `Data utils.py`, `Run real data pipeline.py` and the zip's modules.

Data utils.py

PYTHON MODULE

Contains

Four helper functions.

Purpose

Merges PurpleAir and EPA on timestamp; adds lag-1 temperature, humidity and pressure (per sensor if asked); draws a random contiguous 3–10-day test block for leakage-resistant splits; returns X/y arrays.

Inputs

DataFrames passed in.

Outputs

DataFrames.

Relation

The same as `data_utils.py` in the zip. With a space in the file name, `import data_utils` cannot find it.

Algorithm1.py

PYTHON SCRIPT

Contains

Driver script, about 120 lines.

Purpose

Loads EPA, tracts and all sensors, resamples to hourly, keeps sensors with 120+ hours, picks the sensor closest to the EPA monitor, and runs Algorithm 1 with contiguous time-block splits. It saves everything Algorithm 2 needs as checkpoint CSVs.

Inputs

Paths from `pipeline_config.py`.

Outputs

`checkpoints/`: `epa_data.csv`, `tract_centroids.csv`, `population.csv`,
`sensor_hour_counts.csv`, `all_sensors_filtered.csv`, `closest_sensor_id.txt`,
`log_linear_params.csv`, `r2_comparison.csv`, `synchronized_calibration_data.csv`.

Relation

The same as `algorithm1.py` in `Final_experiment_package.zip`, minus three path lines. It imports `pipeline_config`, `real_data_loaders`, `calibration_models` and `algorithm1_model_comparison`, which exist only inside `final_experiment.zip`.

Run real data pipeline.py

PYTHON SCRIPT

Contains

One-shot driver for Algorithms 1-3.

Purpose

Runs calibration, per-tract baseline PM2.5, Algorithm 2 (bootstrap $m=10$) and Algorithm 3 ($m=5$) in one go, and prints city-wide totals.

Inputs

`LA_Site_1103.csv`, `la_city_tracts_final.csv`, `City_Boundary.geojson`. Its sensor directory is still set to `mock_sensors`, and the baseline/event windows are July 2025 and Aug 1-20, 2025, not the January fire.

Outputs

Console output only.

Relation

The same as `run_real_data_pipeline.py` in the zip. It needs the zip's modules.

Data files 13

`_sensor_list.csv`

CSV, 260 ROWS

Contains

`sensor_index`, `name`, `latitude`, `longitude` for the sensors inside the LA city limits.

Purpose

Record of which sensors `download_data.py` tried to download.

Relation

Output of `download_data.py`. 259 IDs match LA-city-files; the list has 56087 (not in LA-city-files) and LA-city-files has 292635 (not in the list).

`_sensor_date_range_report.csv`

CSV, 261 ROWS

Contains

`file`, `n_rows`, `min_date`, `max_date` per sensor file; 68 files have 0 rows.

Purpose

Audit of each sensor file's date range.

Relation

Output of `sensor_checker.py` (code2). It lists 261 files including `sensor_56087_history.csv`, which is no longer in LA-city-files; the other 260 row counts match LA-city-files exactly.

`164317 2025-07-01 2025-07-08 60-Minute Average.csv`

CSV, 168 ROWS

Contains

Hourly PurpleAir export for sensor 164317, July 1–7, 2025 (UTC), with 27 fields including A/B channels, VOC, scattering, visibility, `pm2.5_atm` and `pm2.5_cf_1`.

Purpose

Sensor data for the early calibration test.

Relation

Input to `chat_linreg.py`, paired with EPA `Site_4002.csv`.

251757 2025-01-01 2025-03-01 60-Minute Average.csv

CSV, 391 ROWS

Contains

Hourly readings for sensor 251757 (sensor #1 in Sensors.csv), Jan 14 to Feb 28, 2025, with `pm2.5_atm` and `pm2.5_cf_1`.

Purpose

One per-sensor download.

Relation

The `0G Sensor (2)` folder's log records this exact download (391 rows), and that folder has no CSV, so this file appears to have been moved out of it.

average_pm25_by_timestamp.csv

CSV, 1,416 ROWS

Contains

`time_stamp`, `avg_pm25`, `n_sensors` for every hour, Jan 1 to Feb 28, 2025; 95–183 sensors reporting per hour.

Purpose

City-wide raw PurpleAir average.

Relation

Output of `averagePA.py`. The values (roughly 45–195 $\mu\text{g}/\text{m}^3$) are far above the EPA readings for the same hours.

City_Boundary.geojson

GEOJSON

Contains

One polygon feature (properties `OBJECTID`, `CITY`) in CRS84 lon/lat.

Purpose

Outline of the City of Los Angeles, with holes for enclaves.

Relation

Used by `download_data.py`, `getdata.py`, `getLApoinst.py`, Algorithm 4 and the modular pipeline.

daily_pm25.csv

CSV, 264 ROWS

Contains

date, daily_mean, n_obs, fire for 2025-01-01 to 09-30; 25 days flagged as fire.

Purpose

Daily EPA Site 1103 averages with a fire-day flag.

Relation

Its columns match the daily table in EPA_mortality_estimate.py (before its date filter), but no script here writes this file. Values match epa_pm25_daily_avg.csv .

epa_pm25_daily_avg.csv

CSV, 59 ROWS

Contains

timestamp, avg_pm25 , Jan 1 to Feb 28, 2025.

Purpose

Daily EPA averages.

Relation

Output of findwildfire.py .

Experiment1_Summary.csv

CSV, 3 ROWS

Contains

For the closest, 3 km and 5 km datasets: best epoch (195 / 180 / 170), linear R^2 (0.740 / 0.745 / 0.613), MLP R^2 (0.831 / 0.839 / 0.802), and MSEs. All are in-sample, on the log1p scale.

Purpose

Experiment 1 results.

Relation

Output of final_presentation.py / final_presentation2.py .

la_city_tracts_final.csv

CSV, 1,108 ROWS

Contains

GE0ID, NAME, population, centroid_lat, centroid_lon, x_km, y_km; total population 3,840,776.

Purpose

Census tracts whose centroid is inside LA city, with km coordinates relative to the EPA monitor.

Relation

Output of `getdata.py`. Input to `EPA_mortality_estimate.py`, Algorithms 2–4 and the modular pipeline.

la_tract_population.json

JSON

Contains

A Census API response: a header row plus 2,498 LA County tracts, with `NAME`, `B01003_001E` (ACS total population), `state`, `county` and `tract`.

Purpose

Tract population source.

Relation

Input to `getdata.py`.

LA_Site_1103.csv

CSV, 6,220 ROWS

Contains

EPA AQS hourly PM2.5 (parameter 88101, POC 3, Met One BAM-1020 FEM) for site 06-037-1103 at 34.06659, –118.22688, Jan 1 to Sep 30, 2025. Values range from –0.9 to 301.1 $\mu\text{g}/\text{m}^3$.

Purpose

The reference monitor the PurpleAir sensors are calibrated against.

Relation

Used by nearly every LA script. It was likely produced by `LA_EPA_files.py` (same naming pattern).

model_predictions.csv

CSV, 99,803 ROWS (19.7 MB)

Contains

Hourly rows for 101 sensors, Jan 1 to Feb 28, 2025: `sensor_num`, `distance_km` (the column appears twice), In PurpleAir PM2.5, weather and lags, `lat/lon`, `ln_pm25_epa`, `ln_pm25_epa_predicted`, `residual`.

Purpose

Per-row predictions from a log-linear model.

Relation

No script in the folder writes this file. Its columns (`sensor_num`, `distance_km`, `lag` features) resemble the panel built in `LA_full_models.py` / `LA_experiments.py`.

Figures (PNG) 25

Each image was opened and looked at. “Appears to be” means the plot matches a script that only displays it on screen, so it was probably saved by hand.

FILE	WHAT IT SHOWS	MADE BY
<code>closest_epoch_search.png</code>	Test MSE vs epoch (5–200) for the closest-sensor dataset; the curve flattens after about 50 epochs.	<code>final_presentation.py</code>
<code>3km_epoch_search.png</code>	The same for sensors within 3 km.	<code>final_presentation.py</code>
<code>5km_epoch_search.png</code>	The same for sensors within 5 km; noisier, with its minimum near 170.	<code>final_presentation.py</code>
Average Test MSE vs Epoch (version 1).png	MLP test MSE vs epochs 50–1,000 (50 replications); flattens around 65 after 300.	Earlier run of <code>LA_experiments.py</code> Experiment 3
Average Test MSE vs Epoch (version 2).png	The same plot with a higher MSE level (131 falling to 87).	Earlier run of <code>LA_experiments.py</code>
Average Test MSE vs Epoch (version 3).png	The same plot, minimum about 62 at 300 epochs.	Earlier run of <code>LA_experiments.py</code> ; it looks identical to <code>experiment3_mse_vs_epochs.png</code>

FILE	WHAT IT SHOWS	MADE BY
experiment3_mse_vs_epochs.png	MLP average test MSE vs epochs 50–750.	LA_experiments.py
acms_surp_Q-Qplot.png	Q-Q plot of linear-model residuals ($\mu\text{g}/\text{m}^3$), with heavy tails.	Appears to be linear_regression_model.py
acms_surp_Q-Qplot2.png	Q-Q plot of log-linear residuals.	Appears to be loglinear_regression_model.py
acms_surp_cooks-distance.png	Residuals vs leverage (bubble = Cook's distance), linear model.	Appears to be linear_regression_model.py
acms_surp_cooks-distance2.png	The same for the log-linear model.	Appears to be loglinear_regression_model.py
acms_surp_residues-vs-fitted.png	Residuals vs predicted, linear model.	Appears to be linear_regression_model.py
acms_surp_residues-vs-fitted2.png	Residuals vs predicted, log-linear model.	Appears to be loglinear_regression_model.py
acms_surp_scale-location.png	Scale-location plot, linear model.	Appears to be linear_regression_model.py
acms_surp_scale-location2.png	Scale-location plot, log-linear model.	Appears to be loglinear_regression_model.py
average_pm25_by_timestamp.png	City-wide average raw PurpleAir PM2.5 by hour, Jan 1 to Feb 28, 2025; peak about 195 near Jan 9.	averagePA.py
calibration_before_after.png	Neural-process prediction vs EPA, before and after calibration, with a 1:1 line.	No script saves this name; matches the idea in LA_area_model.py step 12
comparisons.png	EPA vs linear, log-linear and ML log-linear predictions, July 1–7, 2025 (“Raw Time Series”).	presentation_code.py (saved from the screen)
model_comparison.png	The same comparison, titled “Hourly Averages”.	An earlier version of presentation_code.py

FILE	WHAT IT SHOWS	MADE BY
epa_missing_diagnostic.png	Missing EPA hours per day (mostly Feb 22-26) and by hour of day.	LA_area_model.py
epa_pm25_daily_avg.png	EPA Site 1103 daily average; peak about 93 $\mu\text{g}/\text{m}^3$ around Jan 8, with a gap near Feb 23-25.	findwildfire.py
epa_pm25_hourly.png	EPA Site 1103 hourly readings; spike to about 300 on Jan 8.	findwildfire.py
logy_vs_individual_variables.png	$\ln(\text{EPA})$ vs $\ln(\text{PurpleAir PM2.5})$, humidity, pressure and temperature, each with a fitted line (slope 0.65 for PM2.5).	loglinear_regression_model.py
mlp_vs_log_lin.png	True vs predicted EPA PM2.5 for log-linear, standard MLP and deep FC network.	week2_draftcode.py (code2)
np_predicted_vs_epa.png	Neural-process prediction at the EPA site vs EPA readings; under-predicts high values.	LA_NP.py / LA_area_model.py

Zip archives ³

All LA PurpleAir sensors.zip

ZIP, 261 CSVs

Contains

An earlier download of `sensor_<id>_history.csv` files covering 2024-12-01 to 2025-02-01 (115 files start Dec 1, 2024). 97 files are empty. It includes sensor 56087. Columns are `time_stamp`, `humidity`, `temperature`, `pressure`, `pm2.5_cf_1`.

Purpose

Archive of an older, differently dated download.

Relation

Not the same data as LA-city-files: the 192 non-empty current files differ in date range and column order. The 68 empty files in LA-city-files are identical to empty files here, so they appear to be left over from this older download (`download_data.py` does not write files for empty sensors).

final_experiment.zip

ZIP, 19 FILES

Contains

The modular Python implementation of Algorithms 1–4 with a README. Each file is described in the [final_experiment.zip contents](#) section below.

Purpose

Packaged, staged version of the mortality pipeline.

Relation

Source of `config.py`, `Data utils.py` and `Run real data pipeline.py` in `code1`. The modules that `Algorithm1.py` imports live only here.

Final_experiment_package.zip

ZIP, 4 FILES

Contains

`algorithm1.py`, `stage3_baseline_pm25.py`, `stage4_algorithm2_loglinear.py`,
`stage5_algorithm3_mlp.py`.

Purpose

Copies of four scripts from `final_experiment.zip`, each with three extra lines that add the script's folder to Python's import path.

Relation

Apart from those three lines, `algorithm1.py` is identical to `Algorithm1.py` in `code1`, and the three stage files match their counterparts in `final_experiment.zip`.

final_experiment.zip contents ¹⁹

A modular implementation of Algorithms 1–4, designed to run as five stages that pass data through CSV checkpoints in `checkpoints/`.

FILE	WHAT IT DOES
README.md	Explains the file layout, the synthetic-data quick start, the recommended five-stage pipeline, the LA data sources and a thin-plate-spline fix. It lists an <code>algorithm4_visualization.py</code> stub that is not in the zip.
requirements.txt	numpy, pandas, scipy, scikit-learn, statsmodels, tensorflow.
config.py	Shared constants (identical to <code>code1/config.py</code>).
pipeline_config.py	Paths (sensor folder <code>purpleair_sensors</code> , metadata <code>sensor_metadata.csv</code>), the 120-hour filter, the baseline window (July 2025) and event window (Aug 1–20, 2025), replicate counts, and a <code>ckpt()</code> helper for <code>checkpoints/</code> .

FILE	WHAT IT DOES
data_utils.py	Sync, lag features and contiguous time-block split (identical to code1/Data utils.py).
real_data_loader.py	Loaders for the EPA file (filters to parameter 88101 and a single POC), tracts, the city boundary and PurpleAir files. Columns are auto-detected, temperature is converted from °F to °C, timestamps are converted to naive local time, and lat/lon are attached from a metadata CSV. Also: an equirectangular km projection matching la_city_tracts_final.csv, hourly resampling, the 120-hour filter and a closest-sensor finder.
calibration_models.py	Log-linear OLS fit/predict/save/load (statsmodels) and a Keras MLP (128-BN-Dropout 0.3-64-32) that predicts raw EPA PM2.5, with early stopping.
thin_plate_spline.py	Its own penalised thin-plate spline ($\phi(r) = r^2 \log r$, $\lambda = 1e-3$) solved as a linear system, described as matching the source paper's Eq. 6.
algorithm1_model_comparison.py	Syncs the data, adds lags, fits the full-data log-linear model and runs N paired log-linear vs MLP comparisons on contiguous time-block splits; includes a synthetic smoke test.
mortality_utils.py	Daily spline interpolation to tracts, RR/AF/M, per-tract baseline PM, GBD daily Bd, $T = U + B$, 95% CI and the uniform EPA-only estimate.
algorithm2_loglinear.py	Point estimate plus 50-replicate bootstrap of the calibration data, and the EPA-only comparison.
algorithm3_mlp.py	Ensemble of m retrained MLPs: M_{mean} , U_{mean} , $B = \text{Var}(M)$, T , CI .
run_pipeline.py	Runs Algorithms 1-3 on synthetic data. It imports algorithm4_visualization, which is not in the zip, so it stops at that import.
run_real_data_pipeline.py	One-shot real-data driver (identical to code1/Run real data pipeline.py).
stage1_load_and_filter_sensors.py	Stage 1: loads EPA, tracts and sensors, resamples, applies the 120-hour filter, writes checkpoints.
stage2_algorithm1_calibration.py	Stage 2: closest sensor + Algorithm 1; saves coefficients, R^2 and synchronized data.

FILE	WHAT IT DOES
stage3_baseline_pm25.py	Stage 3: calibrates baseline-window readings, interpolates to tracts, and averages to a per-tract baseline PM2.5.
stage4_algorithm2_loglinear.py	Stage 4: Algorithm 2 over the event window; writes point estimate, bootstrap replicates and EPA-only estimate.
stage5_algorithm3_mlp.py	Stage 5: Algorithm 3 MLP ensemble; writes the ensemble estimate and per-run results.

code2_zip_short 28 files

Sensor-checking scripts, sensor lists and reports, the Illinois EPA test data, one figure, the neural-process weights, and census tract shapefiles.

Scripts 5

sensor_checker.py

PYTHON SCRIPT

Contains

Docstring calls it `sensor_date_range_check.py`.

Purpose

Reports each sensor file's first and last timestamp, finds the most common date range, and lists files whose range differs. The aim is to catch leftovers from an older download.

Inputs

`sensor*_history.csv` in the script's folder.

Outputs

`_sensor_date_range_report.csv` (the copy is in code1).

Relation

Run on the LA-city-files set.

sensor_list.py

PYTHON SCRIPT

Contains

About 60 lines.

Purpose

Counts hours per sensor file and lists sensors with any data and with at least 120 hours.

Inputs

sensor*_history.csv .

Outputs

sensors_with_data.csv , sensors_with_at_least_120_hours.csv .

Relation

The 120-hour threshold is the same filter used in Algorithms 1-3 and Phase1.py.

sensor_sorter.py

PYTHON SCRIPT

Contains

Docstring calls it `sort_purpleair_files.py` . **By default it overwrites the files in place.**

Purpose

Sorts every sensor file by timestamp, and reports empty files and duplicate timestamps.

Inputs

sensor*_history.csv in the given folder (or the current one).

Outputs

The same files, rewritten.

Relation

All 192 non-empty LA-city-files files are in time order with no duplicate hours.

test.py

PYTHON SCRIPT

Contains

Two lines.

Purpose

Prints the path of the Python interpreter being used.

Outputs

Console output.

Relation

Environment check only.

week2_draftcode.py

PYTHON SCRIPT

Contains

scikit-learn models plus a comparison plot.

Purpose

Merges EPA (GMT) with PurpleAir on timestamp and compares three models on 80/20 splits: log-linear regression, a standard MLP (64-32-16) on raw values, and a deep network (256-128-64-32-16) on log values. Reports RMSE, R^2 and adjusted R^2 for each.

Inputs

Site_4002.csv , sensor_data.csv .

Outputs

mlp_vs_log_lin.png (the copy is in code1).

Relation

Extends first_machine_model.py (code1).

y_vs_individual_variables.png

PNG IMAGE

Contains

A 2×2 grid of scatter plots of EPA PM2.5 (Illinois Site 4002, July 1–7, 2025) against each predictor, each with a fitted line. **PurpleAir PM2.5 (pm2.5_cf_1)**: a strong linear pattern, fit $y = 0.59x + 3.74$, with a few outliers (e.g. EPA ≈ 67 at PurpleAir ≈ 30 , and EPA ≈ 27 at PurpleAir ≈ 70). **Humidity**: weak negative trend, $y = -0.13x + 19.41$ (25–58%). **Pressure**: weak positive trend, $y = 1.91x - 1879.45$ (986–996 hPa), with the high EPA values bunched near 992–993 hPa. **Temperature**: essentially flat, $y = 0.01x + 13.12$ (77–106°F).

Purpose

Shows that PurpleAir PM2.5 is the one strong predictor of the EPA reading; the weather variables add little on their own.

Relation

Output of `linear_regression_model.py` (code1). It's the raw-scale companion to `logy_vs_individual_variables.png` (code1), whose PM2.5 slope on the log scale is 0.65.

Site_4002.csv

CSV, 6,535 ROWS

Contains

EPA AQS hourly PM2.5 for site 17-043-4002, DuPage County, **Illinois** (Teledyne T640), Jan 1 to Sep 30, 2025.

Purpose

Reference data for the early calibration tests (not LA).

Relation

Used by `chat_linreg.py`, `linear/loglinear_regression_model.py`, `first_machine_model.py`, `presentation_code.py` and `week2_draftcode.py`.

sensor_data.csv

CSV, 168 ROWS

Contains

Hourly `time_stamp`, `humidity`, `temperature`, `pressure`, `pm2.5_cf_1` for July 1–7, 2025, at UTC–05:00. The file doesn't record the sensor ID.

Purpose

PurpleAir readings paired with Site 4002.

Relation

Used with `Site_4002.csv` by the early calibration scripts.

Sensors.csv

CSV, 137 ROWS

Contains

`Sensor Num`, `Sensor ID`, `Latitude`, `Longitude`; sensors numbered 1–137.

Purpose

Master list for the 137-sensor LA study.

Relation

Read by `LA_full_models.py`, `LA_experiments.py`, `LA_NP.py` and `LA_area_model.py`. It matches the downloads in **OG Sensor Files**. Only 62 of its IDs are in the 260-sensor city list.

Sensors + coordinates - Sheet1.csv

CSV, 260 ROWS

Contains

`sensor`, `lat`, `lon`; only 1 row has coordinates filled in.

Purpose

Spreadsheet export of the 260 sensor IDs before lookup.

Relation

Input to `GPSfinder.py`.

Sensors_with_coordinates.csv

CSV, 260 ROWS

Contains

sensor, lat, lon, all filled in.

Purpose

Coordinates for the 260 city sensors.

Relation

Output of GPSfinder.py. It covers every ID in LA-city-files. Used by final_presentation(2).py, Algorithm 2 and the modular pipeline.

sensors_by_distance.csv

CSV, 137 ROWS

Contains

sensor_num, sensor_id, distance_km, rows_found, completeness_pct, status : 125 EXCLUDED, 4 OK, 6 PARTIAL, 2 EMPTY FILE.

Purpose

Sensor completeness report sorted by distance to EPA.

Relation

Output of LA_full_models.py with its current exclusion list.

sensors_by_distance_saved.csv

CSV, 137 ROWS

Contains

The same columns with no exclusions: 24 OK, 78 PARTIAL, 35 EMPTY FILE.

Purpose

Saved copy of an earlier report.

Relation

Same numbers as sensor_diagnostic_report.csv (only number formatting differs).

sensor_diagnostic_report.csv

CSV, 137 ROWS

Contains

The same columns and statuses as sensors_by_distance_saved.csv, with IDs stored as decimals (e.g. 208493.0).

Purpose

Completeness report for all 137 sensors. Sensor 208493 (#19) is the closest to EPA, at 2.15 km.

Relation

The same structure as LA_full_models.py's output, run with no exclusions.

sensors_with_data.csv

CSV, 192 ROWS

Contains

file, n_hours for sensor files with at least one row.

Purpose

Output of sensor_list.py.

Relation

Matches the 192 non-empty LA-city-files files.

sensors_with_at_least_120_hours.csv

CSV, 191 ROWS

Contains

file, n_hours for files with 120+ hours.

Purpose

Output of sensor_list.py.

Relation

The sensors that pass the 120-hour filter in Algorithms 2-3. Only sensor_260863 (15 hours) drops out.

spatial_np_state_dict.pt

PYTORCH WEIGHTS

Contains

A saved `state_dict` : encoder layers (3 → 64 → 64 → 32) and decoder layers.

Purpose

Trained spatial neural-process model.

Relation

Output of `LA_NP.py` or `LA_area_model.py` (both save this name; the architecture matches).

Census tract shapefiles ¹¹

tl_2023_06_tract.zip

ZIP, SHAPEFILE

Contains

US Census TIGER/Line 2023 census tracts for California: 9,129 tracts (.shp, .shx, .dbf, .prj, .cpg, two .iso.xml metadata files).

Purpose

Tract geometry.

Relation

Read directly (without unzipping) by `getdata.py`.

tl_2024_06_tract.zip

ZIP, SHAPEFILE

Contains

TIGER/Line 2024 California tracts (9,129).

Purpose

Tract geometry for maps.

Relation

The loose `tl_2024_06_tract.*` files below are byte-for-byte copies of this zip's contents.

tl_2024_06_tract.shp

SHAPEFILE GEOMETRY (51.6 MB)

Contains

Polygon shapes of the 9,129 California tracts.

Purpose

Tract polygons.

Relation

Read by `Algorithm 4 visualization.py` together with the .shx, .dbf and .prj files.

tl_2024_06_tract.shx

SHAPEFILE INDEX

Contains

Offsets into the .shp file.

Purpose

Required companion file.

Relation

Part of the tl_2024_06_tract set.

tl_2024_06_tract.dbf

SHAPEFILE ATTRIBUTES

Contains

One record per tract: STATEFP, COUNTYFP, TRACTCE, GEOID, GEOIDFQ, NAME, NAMELSAD, MTFCC, FUNCSTAT, ALAND, AWATER, INTPTLAT, INTPTLON.

Purpose

Tract IDs and areas.

Relation

Algorithm 4 filters on STATEFP 06 / COUNTYFP 037 and joins on GEOID.

tl_2024_06_tract.prj

PROJECTION FILE

Contains

GCS North American 1983 (NAD83).

Purpose

Coordinate reference system.

Relation

Part of the tl_2024_06_tract set.

tl_2024_06_tract.cpg

ENCODING FILE

Contains

The text "UTF-8".

Purpose

Character encoding of the .dbf.

Relation

Part of the tl_2024_06_tract set.

tl_2024_06_tract.shp.iso.xml

XML METADATA

Contains

ISO 19115 metadata from the U.S. Census Bureau Geography Division.

Purpose

Dataset documentation.

Relation

Not read by any script.

tl_2024_06_tract.shp.ea.iso.xml

XML METADATA

Contains

ISO feature catalogue (“Feature Catalog for the 2024 Census Tracts TIGER/Line Shapefiles”) describing each attribute.

Purpose

Attribute documentation.

Relation

Not read by any script.

tl_2020_06_tract_LACounty.zip

ZIP, SHAPEFILE

Contains

2020 tracts for LA County only (2,498 tracts), including .sbn/.sbx spatial indexes.

Purpose

Alternative tract geometry.

Relation

Not referenced by any script.

tl_2024_37_tract.zip

ZIP, SHAPEFILE

Contains

TIGER/Line 2024 tracts for state FIPS 37 (**North Carolina**), 2,672 tracts.

Purpose

Tract geometry for another state.

Relation

Not referenced by any script.

LA-city-files 260 files

One CSV per PurpleAir sensor inside the City of LA, named `sensor_<id>_history.csv`. These are the files `download_data.py` writes, and the main sensor input to Algorithms 1-3, `final_presentation.py` and the QC scripts.

- **Contents:** hourly readings for 2025-01-01 00:00 to 2025-02-28 23:00 Pacific time (at most 1,416 hours), with columns `time_stamp`, `pm2.5_cf_1`, `humidity`, `temperature`,

pressure . Temperature is in °F.

- **192 files have data** (214,112 rows in total); all are in time order with no duplicate hours. 191 have at least 120 hours. **68 are empty** (header only, with a different column order). They match empty files in `All LA PurpleAir sensors.zip` , so they appear to be left over from the older download.
- Every sensor ID here has coordinates in `Sensors_with_coordinates.csv` . The row counts match `_sensor_date_range_report.csv` .

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_1970_history.csv	873		01-23 15	02-28 23	partial
sensor_2649_history.csv	1,408		01-01 00	02-28 23	partial
sensor_2946_history.csv	1,242		01-02 14	02-28 23	partial
sensor_4054_history.csv	0		-	-	empty
sensor_4417_history.csv	1,416		01-01 00	02-28 23	complete
sensor_5804_history.csv	1,416		01-01 00	02-28 23	complete
sensor_5810_history.csv	785		01-01 00	02-28 23	partial
sensor_6170_history.csv	1,416		01-01 00	02-28 23	complete
sensor_6220_history.csv	1,155		01-11 21	02-28 23	partial
sensor_6472_history.csv	1,406		01-01 00	02-28 23	partial
sensor_7290_history.csv	1,411		01-01 00	02-28 23	partial
sensor_8862_history.csv	1,416		01-01 00	02-28 23	complete
sensor_9168_history.csv	1,407		01-01 00	02-28 23	partial
sensor_9274_history.csv	1,416		01-01 00	02-28 23	complete
sensor_10544_history.csv	1,408		01-01 00	02-28 23	partial
sensor_11006_history.csv	0		-	-	empty
sensor_12272_history.csv	1,414		01-01 00	02-28 23	partial
sensor_13967_history.csv	1,416		01-01 00	02-28 23	complete
sensor_15901_history.csv	1,416		01-01 00	02-28 23	complete
sensor_17793_history.csv	1,416		01-01 00	02-28 23	complete

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_20700_history.csv	1,273		01-01 00	02-28 23	partial
sensor_22567_history.csv	1,416		01-01 00	02-28 23	complete
sensor_23635_history.csv	1,416		01-01 00	02-28 23	complete
sensor_26479_history.csv	1,416		01-01 00	02-28 23	complete
sensor_27109_history.csv	1,416		01-01 00	02-28 23	complete
sensor_28529_history.csv	1,416		01-01 00	02-28 23	complete
sensor_29697_history.csv	1,413		01-01 00	02-28 23	partial
sensor_32339_history.csv	1,416		01-01 00	02-28 23	complete
sensor_32969_history.csv	1,416		01-01 00	02-28 23	complete
sensor_33181_history.csv	1,293		01-01 00	02-28 23	partial
sensor_34481_history.csv	1,411		01-01 00	02-28 23	partial
sensor_34517_history.csv	1,247		01-01 00	02-28 23	partial
sensor_35027_history.csv	1,377		01-01 00	02-28 23	partial
sensor_36143_history.csv	1,022		01-01 00	02-19 07	partial
sensor_36359_history.csv	1,229		01-01 00	02-28 23	partial
sensor_36717_history.csv	1,416		01-01 00	02-28 23	complete
sensor_37217_history.csv	842		01-01 00	02-05 13	partial
sensor_37551_history.csv	1,416		01-01 00	02-28 23	complete
sensor_37885_history.csv	1,410		01-01 00	02-28 23	partial

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_38399_history.csv	1,393		01-01 00	02-28 23	partial
sensor_38649_history.csv	1,415		01-01 00	02-28 23	partial
sensor_40449_history.csv	1,414		01-01 00	02-28 23	partial
sensor_46481_history.csv	155		01-08 14	02-17 18	partial
sensor_48397_history.csv	1,416		01-01 00	02-28 23	complete
sensor_49327_history.csv	1,416		01-01 00	02-28 23	complete
sensor_51323_history.csv	1,408		01-01 00	02-28 23	partial
sensor_55503_history.csv	1,356		01-01 00	02-28 23	partial
sensor_55537_history.csv	1,299		01-01 00	02-25 02	partial
sensor_56109_history.csv	1,313		01-01 00	02-28 23	partial
sensor_56207_history.csv	1,208		01-01 00	02-27 17	partial
sensor_56239_history.csv	1,416		01-01 00	02-28 23	complete
sensor_57515_history.csv	1,415		01-01 00	02-28 23	partial
sensor_64765_history.csv	1,408		01-01 00	02-28 23	partial
sensor_67241_history.csv	1,416		01-01 00	02-28 23	complete
sensor_71813_history.csv	1,415		01-01 00	02-28 23	partial
sensor_73135_history.csv	1,401		01-01 00	02-28 23	partial
sensor_73229_history.csv	1,307		01-01 00	02-28 23	partial
sensor_74069_history.csv	1,416		01-01 00	02-28 23	complete

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_75133_history.csv	156		02-22 12	02-28 23	partial
sensor_76371_history.csv	0		-	-	empty
sensor_76627_history.csv	0		-	-	empty
sensor_76759_history.csv	1,416		01-01 00	02-28 23	complete
sensor_77097_history.csv	0		-	-	empty
sensor_78091_history.csv	1,414		01-01 00	02-28 23	partial
sensor_78373_history.csv	0		-	-	empty
sensor_78635_history.csv	1,372		01-01 00	02-28 23	partial
sensor_79085_history.csv	1,416		01-01 00	02-28 23	complete
sensor_81219_history.csv	1,377		01-01 00	02-28 23	partial
sensor_81663_history.csv	1,414		01-01 00	02-28 23	partial
sensor_81731_history.csv	1,357		01-01 00	02-28 23	partial
sensor_82071_history.csv	1,416		01-01 00	02-28 23	complete
sensor_82363_history.csv	1,416		01-01 00	02-28 23	complete
sensor_82609_history.csv	1,212		01-01 00	02-28 23	partial
sensor_83089_history.csv	1,416		01-01 00	02-28 23	complete
sensor_83465_history.csv	1,401		01-01 00	02-28 23	partial
sensor_84069_history.csv	1,416		01-01 00	02-28 23	complete
sensor_84101_history.csv	158		01-01 00	01-07 13	partial

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_84323_history.csv	1,409		01-01 00	02-28 23	partial
sensor_84627_history.csv	1,183		01-10 17	02-28 23	partial
sensor_85359_history.csv	1,416		01-01 00	02-28 23	complete
sensor_85645_history.csv	1,399		01-01 00	02-28 23	partial
sensor_85985_history.csv	1,416		01-01 00	02-28 23	complete
sensor_86071_history.csv	1,416		01-01 00	02-28 23	complete
sensor_86075_history.csv	1,410		01-01 00	02-28 23	partial
sensor_86185_history.csv	1,280		01-01 00	02-28 23	partial
sensor_86559_history.csv	1,416		01-01 00	02-28 23	complete
sensor_87129_history.csv	1,416		01-01 00	02-28 23	complete
sensor_87181_history.csv	1,413		01-01 00	02-28 23	partial
sensor_87355_history.csv	1,412		01-01 00	02-28 23	partial
sensor_88895_history.csv	1,416		01-01 00	02-28 23	complete
sensor_91479_history.csv	0		-	-	empty
sensor_93203_history.csv	1,402		01-01 00	02-28 23	partial
sensor_94397_history.csv	1,416		01-01 00	02-28 23	complete
sensor_96837_history.csv	1,416		01-01 00	02-28 23	complete
sensor_96903_history.csv	1,416		01-01 00	02-28 23	complete
sensor_99345_history.csv	1,223		01-01 00	02-22 14	partial

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_102490_history.csv	796		01-26 20	02-28 23	partial
sensor_108340_history.csv	1,416		01-01 00	02-28 23	complete
sensor_113320_history.csv	1,356		01-01 00	02-28 23	partial
sensor_118601_history.csv	1,404		01-01 00	02-28 23	partial
sensor_118887_history.csv	1,236		01-08 12	02-28 23	partial
sensor_119601_history.csv	0		-	-	empty
sensor_122949_history.csv	1,416		01-01 00	02-28 23	complete
sensor_124627_history.csv	1,416		01-01 00	02-28 23	complete
sensor_124961_history.csv	1,416		01-01 00	02-28 23	complete
sensor_125031_history.csv	1,391		01-01 00	02-28 23	partial
sensor_128709_history.csv	493		02-08 11	02-28 23	partial
sensor_133117_history.csv	1,407		01-01 00	02-28 23	partial
sensor_133597_history.csv	1,388		01-01 00	02-28 23	partial
sensor_135616_history.csv	1,416		01-01 00	02-28 23	complete
sensor_142608_history.csv	1,416		01-01 00	02-28 23	complete
sensor_150020_history.csv	880		01-01 00	02-06 19	partial
sensor_155385_history.csv	985		01-17 07	02-28 23	partial
sensor_155793_history.csv	0		-	-	empty
sensor_175353_history.csv	1,416		01-01 00	02-28 23	complete

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_175415_history.csv	1,408		01-01 00	02-28 23	partial
sensor_175667_history.csv	1,024		01-16 15	02-28 23	partial
sensor_176911_history.csv	1,186		01-10 14	02-28 23	partial
sensor_188495_history.csv	1,416		01-01 00	02-28 23	complete
sensor_198489_history.csv	1,162		01-11 14	02-28 23	partial
sensor_206827_history.csv	1,416		01-01 00	02-28 23	complete
sensor_208491_history.csv	1,318		01-01 00	02-28 23	partial
sensor_208493_history.csv	1,230		01-01 05	02-28 23	partial
sensor_210109_history.csv	1,416		01-01 00	02-28 23	complete
sensor_212477_history.csv	1,397		01-01 00	02-28 23	partial
sensor_212913_history.csv	0		-	-	empty
sensor_216023_history.csv	1,073		01-06 17	02-28 23	partial
sensor_216251_history.csv	1,368		01-01 00	02-28 23	partial
sensor_219899_history.csv	0		-	-	empty
sensor_223265_history.csv	1,416		01-01 00	02-28 23	complete
sensor_224187_history.csv	1,416		01-01 00	02-28 23	complete
sensor_225561_history.csv	1,405		01-01 00	02-28 23	partial
sensor_225879_history.csv	1,396		01-01 00	02-28 23	partial
sensor_239985_history.csv	1,395		01-01 00	02-28 23	partial

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_240281_history.csv	1,416		01-01 00	02-28 23	complete
sensor_240923_history.csv	1,401		01-01 00	02-28 23	partial
sensor_243681_history.csv	1,416		01-01 00	02-28 23	complete
sensor_244053_history.csv	1,261		01-07 11	02-28 23	partial
sensor_244817_history.csv	1,415		01-01 00	02-28 23	partial
sensor_245025_history.csv	1,416		01-01 00	02-28 23	complete
sensor_245029_history.csv	1,416		01-01 00	02-28 23	complete
sensor_249261_history.csv	1,416		01-01 00	02-28 23	complete
sensor_251641_history.csv	1,015		01-17 12	02-28 23	partial
sensor_251647_history.csv	1,104		01-13 16	02-28 23	partial
sensor_251757_history.csv	391		01-14 09	02-28 15	partial
sensor_251927_history.csv	927		01-21 09	02-28 23	partial
sensor_253391_history.csv	0		-	-	empty
sensor_253399_history.csv	874		01-20 14	02-28 23	partial
sensor_253407_history.csv	661		02-01 11	02-28 23	partial
sensor_253411_history.csv	874		01-23 14	02-28 23	partial
sensor_255627_history.csv	1,138		01-12 14	02-28 23	partial
sensor_255637_history.csv	923		01-21 13	02-28 23	partial
sensor_256219_history.csv	0		-	-	empty

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_256271_history.csv	757		01-28 11	02-28 23	partial
sensor_256421_history.csv	1,037		01-16 16	02-28 23	partial
sensor_256435_history.csv	1,092		01-12 11	02-28 23	partial
sensor_256493_history.csv	472		02-09 08	02-28 23	partial
sensor_256643_history.csv	972		01-19 12	02-28 23	partial
sensor_256653_history.csv	699		01-24 15	02-25 13	partial
sensor_256803_history.csv	945		01-20 15	02-28 23	partial
sensor_256839_history.csv	993		01-18 15	02-28 23	partial
sensor_256923_history.csv	730		01-29 14	02-28 23	partial
sensor_256977_history.csv	657		02-01 15	02-28 23	partial
sensor_257141_history.csv	946		01-20 11	02-28 23	partial
sensor_257145_history.csv	516		02-07 12	02-28 23	partial
sensor_257593_history.csv	873		01-23 15	02-28 23	partial
sensor_257599_history.csv	895		01-22 17	02-28 23	partial
sensor_257643_history.csv	581		02-04 08	02-28 23	partial
sensor_257651_history.csv	942		01-20 18	02-28 23	partial
sensor_257799_history.csv	1,014		01-15 20	02-28 23	partial
sensor_257857_history.csv	987		01-18 21	02-28 23	partial
sensor_257879_history.csv	948		01-20 12	02-28 23	partial

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_257941_history.csv	690		01-30 13	02-28 23	partial
sensor_258239_history.csv	727		01-29 17	02-28 23	partial
sensor_258263_history.csv	880		01-23 08	02-28 23	partial
sensor_258282_history.csv	151		02-22 13	02-28 23	partial
sensor_258287_history.csv	445		02-08 11	02-28 23	partial
sensor_258513_history.csv	664		02-01 08	02-28 23	partial
sensor_258541_history.csv	830		01-25 10	02-28 23	partial
sensor_258565_history.csv	0		-	-	empty
sensor_259187_history.csv	595		01-31 08	02-28 23	partial
sensor_259223_history.csv	865		01-23 18	02-28 23	partial
sensor_259233_history.csv	830		01-22 16	02-28 23	partial
sensor_259239_history.csv	850		01-24 12	02-28 23	partial
sensor_259651_history.csv	946		01-20 14	02-28 23	partial
sensor_259915_history.csv	680		01-31 16	02-28 23	partial
sensor_260011_history.csv	568		02-05 08	02-28 23	partial
sensor_260023_history.csv	519		02-05 12	02-28 23	partial
sensor_260081_history.csv	446		02-10 10	02-28 23	partial
sensor_260249_history.csv	459		02-09 21	02-28 23	partial
sensor_260355_history.csv	634		02-02 14	02-28 23	partial

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_260679_history.csv	528		02-07 00	02-28 23	partial
sensor_260723_history.csv	636		02-02 12	02-28 23	partial
sensor_260839_history.csv	705		01-30 11	02-28 23	partial
sensor_260855_history.csv	226		02-19 14	02-28 23	partial
sensor_260863_history.csv	15		02-28 09	02-28 23	<120 h
sensor_261099_history.csv	0		-	-	empty
sensor_261235_history.csv	611		02-03 13	02-28 23	partial
sensor_261309_history.csv	442		02-10 13	02-28 23	partial
sensor_261361_history.csv	0		-	-	empty
sensor_261363_history.csv	443		02-06 14	02-28 23	partial
sensor_261643_history.csv	428		02-10 10	02-28 23	partial
sensor_261651_history.csv	422		02-11 10	02-28 23	partial
sensor_261751_history.csv	432		02-06 19	02-24 18	partial
sensor_261755_history.csv	471		02-09 08	02-28 23	partial
sensor_262261_history.csv	423		02-11 09	02-28 23	partial
sensor_262343_history.csv	0		-	-	empty
sensor_262437_history.csv	208		02-20 08	02-28 23	partial
sensor_262449_history.csv	216		02-19 22	02-28 23	partial
sensor_263027_history.csv	0		-	-	empty

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_265253_history.csv	0		-	-	empty
sensor_265417_history.csv	0		-	-	empty
sensor_265735_history.csv	0		-	-	empty
sensor_266279_history.csv	0		-	-	empty
sensor_267621_history.csv	0		-	-	empty
sensor_267631_history.csv	0		-	-	empty
sensor_267669_history.csv	0		-	-	empty
sensor_268708_history.csv	0		-	-	empty
sensor_269902_history.csv	0		-	-	empty
sensor_270654_history.csv	0		-	-	empty
sensor_271012_history.csv	0		-	-	empty
sensor_271303_history.csv	0		-	-	empty
sensor_272935_history.csv	0		-	-	empty
sensor_273367_history.csv	0		-	-	empty
sensor_274436_history.csv	0		-	-	empty
sensor_275634_history.csv	0		-	-	empty
sensor_276088_history.csv	0		-	-	empty
sensor_279504_history.csv	0		-	-	empty
sensor_281340_history.csv	0		-	-	empty

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_284000_history.csv	0		-	-	empty
sensor_284002_history.csv	0		-	-	empty
sensor_284042_history.csv	0		-	-	empty
sensor_284134_history.csv	0		-	-	empty
sensor_288248_history.csv	0		-	-	empty
sensor_292635_history.csv	0		-	-	empty
sensor_292727_history.csv	0		-	-	empty
sensor_292775_history.csv	0		-	-	empty
sensor_292925_history.csv	0		-	-	empty
sensor_293651_history.csv	0		-	-	empty
sensor_294053_history.csv	0		-	-	empty
sensor_296843_history.csv	0		-	-	empty
sensor_299439_history.csv	0		-	-	empty
sensor_300669_history.csv	0		-	-	empty
sensor_302278_history.csv	0		-	-	empty
sensor_302888_history.csv	0		-	-	empty
sensor_304070_history.csv	0		-	-	empty
sensor_304902_history.csv	0		-	-	empty
sensor_306510_history.csv	0		-	-	empty

FILE	HOURS	COVERAGE	FIRST (MM-DD HH)	LAST	STATUS
sensor_310639_history.csv	0		-	-	empty
sensor_312711_history.csv	0		-	-	empty
sensor_313261_history.csv	0		-	-	empty
sensor_313507_history.csv	0		-	-	empty
sensor_314771_history.csv	0		-	-	empty
sensor_315083_history.csv	0		-	-	empty
sensor_315103_history.csv	0		-	-	empty
sensor_315225_history.csv	0		-	-	empty
sensor_315573_history.csv	0		-	-	empty
sensor_315803_history.csv	0		-	-	empty
sensor_315825_history.csv	0		-	-	empty
sensor_316549_history.csv	0		-	-	empty

OG Sensor Files 137 folders, 273 files

The per-sensor downloads for the 137-sensor study listed in `Sensors.csv`. Each folder holds a PurpleAir download-tool log and, usually, one CSV.

- **Progress_Logs.log** (in all 137 folders): the download tool's summary. It gives the request window (2025-01-01 to 2025-03-01, 60-minute average), the fields (humidity, temperature, pressure, pm2.5_atm, pm2.5_cf_1), rows downloaded, and when it ran (June 25-26, 2026).
- **<id> 2025-01-01 2025-03-01 60-Minute Average.csv** (in 136 folders): hourly time_stamp, humidity, temperature, pressure, pm2.5_atm, pm2.5_cf_1. Unlike LA-city-files, it includes pm2.5_atm. 36 are empty; 101 have 120+ hours.

- **Numbering:** folder OG Sensor (N) holds sensor number N–1 from Sensors.csv (checked for all 135 numbered folders with a CSV). The unnumbered OG Sensor holds #137. OG Sensor (2) has only its log; the log names sensor 251757 (#1, 391 rows), and that CSV is in code1.
- **Relation:** the file names match what LA_models.py and the LA_* scripts load, but those scripts expect folders named PurpleAir Download Sensor N.

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor	219339 2025-01-01 2025-03-01 60-Minute Average.csv	137	0	-	-	empty
OG Sensor (2)	<i>no CSV (log only)</i>	1	0	-	-	moved
OG Sensor (3)	259651 2025-01-01 2025-03-01 60-Minute Average.csv	2	946	01-20 14	02-28 23	partial
OG Sensor (4)	284002 2025-01-01 2025-03-01 60-Minute Average.csv	3	0	-	-	empty
OG Sensor (5)	67241 2025-01-01 2025-03-01 60-Minute Average.csv	4	1,416	01-01 00	02-28 23	complete
OG Sensor (6)	260723 2025-01-01 2025-03-01 60-Minute Average.csv	5	636	02-02 12	02-28 23	partial
OG Sensor (7)	150020 2025-01-01 2025-03-01 60-Minute Average.csv	6	880	01-01 00	02-06 19	partial
OG Sensor (8)	225879 2025-01-01 2025-03-01 60-Minute Average.csv	7	1,396	01-01 00	02-28 23	partial
OG Sensor (9)	315083 2025-01-01 2025-03-01 60-Minute Average.csv	8	0	-	-	empty
OG Sensor (10)	288248 2025-01-01 2025-03-01 60-Minute Average.csv	9	0	-	-	empty
OG Sensor (11)	256421 2025-01-01 2025-03-01 60-Minute Average.csv	10	1,037	01-16 16	02-28 23	partial
OG Sensor (12)	257857 2025-01-01 2025-03-01 60-Minute Average.csv	11	987	01-18 21	02-28 23	partial

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (13)	271303 2025-01-01 2025-03-01 60-Minute Average.csv	12	0	-	-	empty
OG Sensor (14)	124961 2025-01-01 2025-03-01 60-Minute Average.csv	13	1,416	01-01 00	02-28 23	complete
OG Sensor (15)	94397 2025-01-01 2025-03-01 60-Minute Average.csv	14	1,416	01-01 00	02-28 23	complete
OG Sensor (16)	300669 2025-01-01 2025-03-01 60-Minute Average.csv	15	0	-	-	empty
OG Sensor (17)	284042 2025-01-01 2025-03-01 60-Minute Average.csv	16	0	-	-	empty
OG Sensor (18)	310639 2025-01-01 2025-03-01 60-Minute Average.csv	17	0	-	-	empty
OG Sensor (19)	304070 2025-01-01 2025-03-01 60-Minute Average.csv	18	0	-	-	empty
OG Sensor (20)	208493 2025-01-01 2025-03-01 60-Minute Average.csv	19	1,230	01-01 05	02-28 23	partial
OG Sensor (21)	4670 2025-01-01 2025-03-01 60-Minute Average.csv	20	0	-	-	empty
OG Sensor (22)	311123 2025-01-01 2025-03-01 60-Minute Average.csv	21	0	-	-	empty
OG Sensor (23)	76759 2025-01-01 2025-03-01 60-Minute Average.csv	22	1,416	01-01 00	02-28 23	complete
OG Sensor (24)	256435 2025-01-01 2025-03-01 60-Minute Average.csv	23	1,092	01-12 11	02-28 23	partial
OG Sensor (25)	12272 2025-01-01 2025-03-01 60-Minute Average.csv	24	1,414	01-01 00	02-28 23	partial
OG Sensor (26)	4054 2025-01-01 2025-03-01 60-Minute Average.csv	25	0	-	-	empty

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (27)	219787 2025-01-01 2025-03-01 60-Minute Average.csv	26	1,416	01-01 00	02-28 23	complete
OG Sensor (28)	219853 2025-01-01 2025-03-01 60-Minute Average.csv	27	1,383	01-01 00	02-28 23	partial
OG Sensor (29)	219825 2025-01-01 2025-03-01 60-Minute Average.csv	28	1,316	01-01 00	02-28 23	partial
OG Sensor (30)	220457 2025-01-01 2025-03-01 60-Minute Average.csv	29	1,401	01-01 00	02-28 23	partial
OG Sensor (31)	220455 2025-01-01 2025-03-01 60-Minute Average.csv	30	1,411	01-01 00	02-28 23	partial
OG Sensor (32)	220251 2025-01-01 2025-03-01 60-Minute Average.csv	31	1,383	01-01 00	02-28 23	partial
OG Sensor (33)	220477 2025-01-01 2025-03-01 60-Minute Average.csv	32	0	-	-	empty
OG Sensor (34)	184167 2025-01-01 2025-03-01 60-Minute Average.csv	33	710	01-30 10	02-28 23	partial
OG Sensor (35)	184143 2025-01-01 2025-03-01 60-Minute Average.csv	34	1,172	01-01 00	02-27 09	partial
OG Sensor (36)	264009 2025-01-01 2025-03-01 60-Minute Average.csv	35	0	-	-	empty
OG Sensor (37)	264011 2025-01-01 2025-03-01 60-Minute Average.csv	36	0	-	-	empty
OG Sensor (38)	264017 2025-01-01 2025-03-01 60-Minute Average.csv	37	0	-	-	empty
OG Sensor (39)	264015 2025-01-01 2025-03-01 60-Minute Average.csv	38	0	-	-	empty
OG Sensor (40)	220463 2025-01-01 2025-03-01 60-Minute Average.csv	39	164	01-01 00	01-07 19	partial

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (41)	220427 2025-01-01 2025-03-01 60-Minute Average.csv	40	1,192	01-01 00	02-28 23	partial
OG Sensor (42)	184197 2025-01-01 2025-03-01 60-Minute Average.csv	41	1,416	01-01 00	02-28 23	complete
OG Sensor (43)	220491 2025-01-01 2025-03-01 60-Minute Average.csv	42	1,339	01-01 00	02-28 23	partial
OG Sensor (44)	220445 2025-01-01 2025-03-01 60-Minute Average.csv	43	0	-	-	empty
OG Sensor (45)	220595 2025-01-01 2025-03-01 60-Minute Average.csv	44	164	01-01 00	01-07 19	partial
OG Sensor (46)	184183 2025-01-01 2025-03-01 60-Minute Average.csv	45	542	01-01 00	01-23 13	partial
OG Sensor (47)	184159 2025-01-01 2025-03-01 60-Minute Average.csv	46	792	01-01 00	02-04 08	partial
OG Sensor (48)	184133 2025-01-01 2025-03-01 60-Minute Average.csv	47	232	02-19 08	02-28 23	partial
OG Sensor (49)	184161 2025-01-01 2025-03-01 60-Minute Average.csv	48	0	-	-	empty
OG Sensor (50)	184139 2025-01-01 2025-03-01 60-Minute Average.csv	49	160	01-01 00	01-07 15	partial
OG Sensor (51)	184135 2025-01-01 2025-03-01 60-Minute Average.csv	50	0	-	-	empty
OG Sensor (52)	184141 2025-01-01 2025-03-01 60-Minute Average.csv	51	1,416	01-01 00	02-28 23	complete
OG Sensor (53)	184189 2025-01-01 2025-03-01 60-Minute Average.csv	52	1,379	01-01 00	02-27 13	partial
OG Sensor (54)	184177 2025-01-01 2025-03-01 60-Minute Average.csv	53	1,375	01-01 00	02-28 23	partial

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (55)	184175 2025-01-01 2025-03-01 60-Minute Average.csv	54	0	-	-	empty
OG Sensor (56)	184179 2025-01-01 2025-03-01 60-Minute Average.csv	55	1,381	01-01 00	02-27 13	partial
OG Sensor (57)	184169 2025-01-01 2025-03-01 60-Minute Average.csv	56	792	01-01 00	02-04 08	partial
OG Sensor (58)	184131 2025-01-01 2025-03-01 60-Minute Average.csv	57	0	-	-	empty
OG Sensor (59)	184145 2025-01-01 2025-03-01 60-Minute Average.csv	58	160	01-01 00	01-07 15	partial
OG Sensor (60)	286510 2025-01-01 2025-03-01 60-Minute Average.csv	59	0	-	-	empty
OG Sensor (61)	23429 2025-01-01 2025-03-01 60-Minute Average.csv	60	1,416	01-01 00	02-28 23	complete
OG Sensor (62)	23295 2025-01-01 2025-03-01 60-Minute Average.csv	61	1,416	01-01 00	02-28 23	complete
OG Sensor (63)	188345 2025-01-01 2025-03-01 60-Minute Average.csv	62	1,416	01-01 00	02-28 23	complete
OG Sensor (64)	22727 2025-01-01 2025-03-01 60-Minute Average.csv	63	1,416	01-01 00	02-28 23	complete
OG Sensor (65)	224015 2025-01-01 2025-03-01 60-Minute Average.csv	64	1,388	01-01 00	02-28 23	partial
OG Sensor (66)	69409 2025-01-01 2025-03-01 60-Minute Average.csv	65	1,416	01-01 00	02-28 23	complete
OG Sensor (67)	90403 2025-01-01 2025-03-01 60-Minute Average.csv	66	0	-	-	empty
OG Sensor (68)	272935 2025-01-01 2025-03-01 60-Minute Average.csv	67	0	-	-	empty

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (69)	208491 2025-01-01 2025-03-01 60-Minute Average.csv	68	1,318	01-01 00	02-28 23	partial
OG Sensor (70)	244053 2025-01-01 2025-03-01 60-Minute Average.csv	69	1,261	01-07 11	02-28 23	partial
OG Sensor (71)	6220 2025-01-01 2025-03-01 60-Minute Average.csv	70	1,155	01-11 21	02-28 23	partial
OG Sensor (72)	96855 2025-01-01 2025-03-01 60-Minute Average.csv	71	1,416	01-01 00	02-28 23	complete
OG Sensor (73)	74069 2025-01-01 2025-03-01 60-Minute Average.csv	72	1,416	01-01 00	02-28 23	complete
OG Sensor (74)	256653 2025-01-01 2025-03-01 60-Minute Average.csv	73	699	01-24 15	02-25 13	partial
OG Sensor (75)	76627 2025-01-01 2025-03-01 60-Minute Average.csv	74	0	-	-	empty
OG Sensor (76)	86185 2025-01-01 2025-03-01 60-Minute Average.csv	75	1,280	01-01 00	02-28 23	partial
OG Sensor (77)	49327 2025-01-01 2025-03-01 60-Minute Average.csv	76	1,416	01-01 00	02-28 23	complete
OG Sensor (78)	256839 2025-01-01 2025-03-01 60-Minute Average.csv	77	993	01-18 15	02-28 23	partial
OG Sensor (79)	9168 2025-01-01 2025-03-01 60-Minute Average.csv	78	1,407	01-01 00	02-28 23	partial
OG Sensor (80)	155793 2025-01-01 2025-03-01 60-Minute Average.csv	79	0	-	-	empty
OG Sensor (81)	259233 2025-01-01 2025-03-01 60-Minute Average.csv	80	830	01-22 16	02-28 23	partial
OG Sensor (82)	256423 2025-01-01 2025-03-01 60-Minute Average.csv	81	660	01-14 15	02-13 00	partial

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (83)	239985 2025-01-01 2025-03-01 60-Minute Average.csv	82	1,395	01-01 00	02-28 23	partial
OG Sensor (84)	81731 2025-01-01 2025-03-01 60-Minute Average.csv	83	1,357	01-01 00	02-28 23	partial
OG Sensor (85)	29295 2025-01-01 2025-03-01 60-Minute Average.csv	84	1,275	01-01 00	02-28 23	partial
OG Sensor (86)	198489 2025-01-01 2025-03-01 60-Minute Average.csv	85	1,162	01-11 14	02-28 23	partial
OG Sensor (87)	265253 2025-01-01 2025-03-01 60-Minute Average.csv	86	0	-	-	empty
OG Sensor (88)	258282 2025-01-01 2025-03-01 60-Minute Average.csv	87	151	02-22 13	02-28 23	partial
OG Sensor (89)	30737 2025-01-01 2025-03-01 60-Minute Average.csv	88	1,401	01-01 00	02-28 23	partial
OG Sensor (90)	188495 2025-01-01 2025-03-01 60-Minute Average.csv	89	1,416	01-01 00	02-28 23	complete
OG Sensor (91)	261751 2025-01-01 2025-03-01 60-Minute Average.csv	90	432	02-06 19	02-24 18	partial
OG Sensor (92)	99315 2025-01-01 2025-03-01 60-Minute Average.csv	91	1,240	01-01 00	02-28 23	partial
OG Sensor (93)	251641 2025-01-01 2025-03-01 60-Minute Average.csv	92	1,015	01-17 12	02-28 23	partial
OG Sensor (94)	256643 2025-01-01 2025-03-01 60-Minute Average.csv	93	972	01-19 12	02-28 23	partial
OG Sensor (95)	292395 2025-01-01 2025-03-01 60-Minute Average.csv	94	0	-	-	empty
OG Sensor (96)	21293 2025-01-01 2025-03-01 60-Minute Average.csv	95	1,240	01-08 08	02-28 23	partial

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (97)	221399 2025-01-01 2025-03-01 60-Minute Average.csv	96	1,405	01-01 00	02-28 23	partial
OG Sensor (98)	258495 2025-01-01 2025-03-01 60-Minute Average.csv	97	603	02-03 08	02-28 10	partial
OG Sensor (99)	125949 2025-01-01 2025-03-01 60-Minute Average.csv	98	1,375	01-01 00	02-28 23	partial
OG Sensor (100)	83647 2025-01-01 2025-03-01 60-Minute Average.csv	99	901	01-01 00	02-07 12	partial
OG Sensor (101)	78129 2025-01-01 2025-03-01 60-Minute Average.csv	100	1,411	01-01 00	02-28 23	partial
OG Sensor (102)	251647 2025-01-01 2025-03-01 60-Minute Average.csv	101	1,104	01-13 16	02-28 23	partial
OG Sensor (103)	253399 2025-01-01 2025-03-01 60-Minute Average.csv	102	874	01-20 14	02-28 23	partial
OG Sensor (104)	256039 2025-01-01 2025-03-01 60-Minute Average.csv	103	1,400	01-01 16	02-28 23	partial
OG Sensor (105)	55351 2025-01-01 2025-03-01 60-Minute Average.csv	104	1,416	01-01 00	02-28 23	complete
OG Sensor (106)	292775 2025-01-01 2025-03-01 60-Minute Average.csv	105	0	-	-	empty
OG Sensor (107)	133117 2025-01-01 2025-03-01 60-Minute Average.csv	106	1,407	01-01 00	02-28 23	partial
OG Sensor (108)	78635 2025-01-01 2025-03-01 60-Minute Average.csv	107	1,372	01-01 00	02-28 23	partial
OG Sensor (109)	265629 2025-01-01 2025-03-01 60-Minute Average.csv	108	0	-	-	empty
OG Sensor (110)	38649 2025-01-01 2025-03-01 60-Minute Average.csv	109	1,415	01-01 00	02-28 23	partial

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (111)	6472 2025-01-01 2025-03-01 60-Minute Average.csv	110	1,406	01-01 00	02-28 23	partial
OG Sensor (112)	293651 2025-01-01 2025-03-01 60-Minute Average.csv	111	0	-	-	empty
OG Sensor (113)	86071 2025-01-01 2025-03-01 60-Minute Average.csv	112	1,416	01-01 00	02-28 23	complete
OG Sensor (114)	32339 2025-01-01 2025-03-01 60-Minute Average.csv	113	1,416	01-01 00	02-28 23	complete
OG Sensor (115)	269902 2025-01-01 2025-03-01 60-Minute Average.csv	114	0	-	-	empty
OG Sensor (116)	256733 2025-01-01 2025-03-01 60-Minute Average.csv	115	1,028	01-17 04	02-28 23	partial
OG Sensor (117)	239775 2025-01-01 2025-03-01 60-Minute Average.csv	116	1,140	01-12 12	02-28 23	partial
OG Sensor (118)	258541 2025-01-01 2025-03-01 60-Minute Average.csv	117	830	01-25 10	02-28 23	partial
OG Sensor (119)	89673 2025-01-01 2025-03-01 60-Minute Average.csv	118	947	01-15 19	02-28 23	partial
OG Sensor (120)	257879 2025-01-01 2025-03-01 60-Minute Average.csv	119	948	01-20 12	02-28 23	partial
OG Sensor (121)	260355 2025-01-01 2025-03-01 60-Minute Average.csv	120	634	02-02 14	02-28 23	partial
OG Sensor (122)	84627 2025-01-01 2025-03-01 60-Minute Average.csv	121	1,183	01-10 17	02-28 23	partial
OG Sensor (123)	20704 2025-01-01 2025-03-01 60-Minute Average.csv	122	1,396	01-01 00	02-28 23	partial
OG Sensor (124)	296843 2025-01-01 2025-03-01 60-Minute Average.csv	123	0	-	-	empty

FOLDER	CSV INSIDE	SENSOR #	HOURS	FIRST	LAST	STATUS
OG Sensor (125)	257645 2025-01-01 2025-03-01 60-Minute Average.csv	124	936	01-20 22	02-28 23	partial
OG Sensor (126)	96837 2025-01-01 2025-03-01 60-Minute Average.csv	125	1,416	01-01 00	02-28 23	complete
OG Sensor (127)	79953 2025-01-01 2025-03-01 60-Minute Average.csv	126	1,407	01-01 00	02-28 23	partial
OG Sensor (128)	258475 2025-01-01 2025-03-01 60-Minute Average.csv	127	873	01-23 15	02-28 23	partial
OG Sensor (129)	251639 2025-01-01 2025-03-01 60-Minute Average.csv	128	997	01-18 10	02-28 23	partial
OG Sensor (130)	92105 2025-01-01 2025-03-01 60-Minute Average.csv	129	1,400	01-01 00	02-28 23	partial
OG Sensor (131)	60479 2025-01-01 2025-03-01 60-Minute Average.csv	130	1,246	01-01 00	02-28 23	partial
OG Sensor (132)	78587 2025-01-01 2025-03-01 60-Minute Average.csv	131	1,416	01-01 00	02-28 23	complete
OG Sensor (133)	91969 2025-01-01 2025-03-01 60-Minute Average.csv	132	1,413	01-01 00	02-28 23	partial
OG Sensor (134)	91125 2025-01-01 2025-03-01 60-Minute Average.csv	133	0	-	-	empty
OG Sensor (135)	5218 2025-01-01 2025-03-01 60-Minute Average.csv	134	1,416	01-01 00	02-28 23	complete
OG Sensor (136)	22735 2025-01-01 2025-03-01 60-Minute Average.csv	135	1,416	01-01 00	02-28 23	complete
OG Sensor (137)	33855 2025-01-01 2025-03-01 60-Minute Average.csv	136	1,416	01-01 00	02-28 23	complete

.git version history

This is a Git repository with the remote `https://github.com/geethurajan/purple_air_DL.git` . It has 8 commits, all dated 2026-09-28 (“pa code files sept 28”, “Add files via upload”, six “code2 sept 28”). It tracks 634 files: 260 in LA-city-files, 273 in OG Sensor Files, 73 in code1_zip_short and 28 in code2_zip_short. The `.git` folder (about 134 MB) holds Git's own internal object and configuration files, not project content.